# The Timed $P$ Transformation for Distributed Real-Time Systems

José Meseguer[1] and Peter Csaba Ölveczky[2]

[1] University of Illinois, Urbana-Champaign, USA
[2] University of Oslo, Oslo, Norway

**Abstract.** Both *qualitative* and *quantitative* properties are crucially important in most distributed systems. Analyzing these two kinds of properties often requires using two different system models, necessitating substantial efforts in developing and evolving two quite different models and raising the question of their *semantic consistency*. For *untimed* actor models of distributed systems this problem has been solved in [17] by the $P$ *transformation*, which maps a rewrite theory $\mathcal{R}$ modeling an actor system to a *probabilistic* real-time rewrite theory $\mathcal{R}_{\Pi}$ semantically consistent with $\mathcal{R}$ and amenable to quantitative analysis by statistical model checking. This paper extends the $P$ transformation to distributed *real-time* systems communicating by asynchronous message passing. We define a *timed* $P$ transformation whose input theories are real-time rewrite theories that extend timed actor systems. We prove that the timed $P$ transformation enjoys desired semantic properties, including amenability to statistical model-checking analysis of the transformed theory.

## 1 Introduction

Distributed real-time systems communicating by message passing are intrinsically nondeterministic because, besides being concurrent, the delays involved in message transmission are unpredictable. For most of these systems, both *qualitative* properties, such as safety properties, and *quantitative* properties, such as latency and throughput, are crucially important. Analyzing both qualitative and quantitative properties is challenging because the models used for each kind of analysis are often different: a concurrent model will typically be used for verifying qualitative properties by model checking or theorem proving, whereas a probabilistic model such as a stochastic timed automaton may be used to analyze quantitative properties by statistical model checking (see, e.g., [11]). This leads to a *semantic gap* between the different models developed for analyzing different properties, since there may be no clear semantic relation between them.

For *untimed* distributed systems communicating by message passing in a (generalized) version of the actor model [2] the gap has been bridged in [17] by the so-called $P$ transformation. The key idea is to quantify by means of continuous probability distributions $\Pi$ the nondeterminism inherent in message transmission by the underlying network. Since distributed systems can be naturally modeled as rewrite theories [19], which are supported by the Maude language [10], the

inputs to the $P$ transformation are rewrite theories $\mathcal{R}$ formally specifying such distributed systems, and $P$ is a theory transformation $P : \mathcal{R} \mapsto \mathcal{R}_{\Pi}$ mapping $\mathcal{R}$ to a *probabilistic* real-time rewrite theory $\mathcal{R}_{\Pi}$.

The semantic gap between models for qualitative and quantitative analysis is bridged by the $P$ transformation because: (i) all qualitative analysis can be performed on $\mathcal{R}$ itself; (ii) all quantitative analysis, typically by statistical model checking [3,17], is carried out on $\mathcal{R}_{\Pi}$, which is a refinement of $\mathcal{R}$ that quantifies by means of $\Pi$ the nondeterminism inherent in $\mathcal{R}$ and, since it just restricts $\mathcal{R}$'s nondeterministic behaviors, is *semantically consistent* with $\mathcal{R}$ itself [17].

This paper extends the $P$ transformation in [17] to a *timed* $P$ transformation whose inputs are real-time rewrite theories specifying distributed *real-time* systems that significantly extend timed actor systems. This makes the class of input theories very general, to support a wide range of applications.

We provide some prerequisites in §2, and describe in §3 the class of real-time rewrite theories that are the inputs to the timed $P$ transformation, which is defined in §4. In §5 we provide two key semantic properties of the timed $P$ transformation ensuring that for any rewrite theory $\mathcal{R}$ and initial state meeting the input requirements: (i) when $\mathcal{R}_{\Pi}$ is viewed as a nondeterministic system, its behaviors from the given initial state are semantically consistent with those in $\mathcal{R}$; and (ii) $\mathcal{R}_{\Pi}$ is *purely probabilistic* and therefore analyzable by standard statistical model-checking algorithms. By bridging the semantic gap between quantitative and qualitative system analysis we obtain a *unified formal methodology*, outlined in §6, for designing distributed real-time systems in rewriting logic and analyzing them with respect to both their qualitative and quantitative properties. §7 discusses related work, and §8 gives some concluding remarks. We refer to our longer report [21] for more detail, explanations, proofs, and three examples illustrating our methodology.

## 2 Prerequisites

### 2.1 Rewriting Logic, Maude, and Real-Time Rewrite Theories

**Rewriting Logic** [19,20] is a logic well suited to formally specify and program concurrent systems. A *rewrite theory* is a 4-tuple $\mathcal{R} = (\Sigma, E, L, R)$, where:

- $\Sigma$ is an algebraic *signature*, i.e., a set of *sorts*, *subsorts*, and *function symbols*.
- $(\Sigma, E)$ is an *order-sorted equational logic theory* [13] specifying the system's data types, with $E$ a set of (possibly conditional) equations and axioms.
- $L$ is a set of rule *labels*.
- $R$ is a collection of *labeled conditional rewrite rules* $[l] : t \to t'$ **if** *cond*, with $t, t'$ $\Sigma$-terms and $l \in L$, that specify the system's local transitions.

**Maude** [10] is a formal specification language and analysis tool for distributed systems. A Maude module specifies a rewrite theory $\mathcal{R}$ as a *system module* `mod` $\mathcal{R}$ `endm`. We summarize Maude's syntax and refer to [10] for details. Operators

are declared `op` $f$ `:` $s_1 \ldots s_n$ `->` $s$ and can have user-definable syntax, with '`_`' denoting argument positions, as in `_+_`. (Unconditional and conditional) equations and rewrite rules are introduced with, resp., the keywords `eq` and `ceq`, and `rl` and `crl`. Mathematical variables are declared with the keywords `var` and `vars`.

Maude provides *syntactic sugar* to support object-oriented specification: A declaration `class` $C$ `|` $att_1$ `:` $s_1$`,` `...,` $att_n$ `:` $s_n$ declares an *object class* $C$ with attributes $att_1$ to $att_n$ of sorts $s_1$ to $s_n$. An *object* of class $C$ is a term `<` $o$ `:` $C$ `|` $att_1 : val_1, \ldots, att_n : val_n$ `>`, where $o$ (of sort `Oid`) is the object's *identifier*, and $val_1$ to $val_n$ are the current values of the attributes $att_1$ to $att_n$. *Messages* are terms of sort `Msg`. A state of an object-oriented system often has the sort `Configuration` and represents a multiset of objects and messages.

**Real-Time Rewrite Theories.** Real-time systems can be modeled in rewriting logic as real-time rewrite theories [24], which can be analyzed in Maude [7]. Such theories contain a data type `Time` of (discrete or continuous) time values. Their rules are ordinary rewrite rules, which model *instantaneous change*, and *tick* rewrite rules `[`*tick*`]` `:` `{`$t$`}` $\rightarrow$ `{`$t'$`}` `in time` $\tau$ *if cond* which model *time advance*: the system evolves from state `{`$t$`}` to state `{`$t'$`}` in time $\tau$. The states have the form `{`$u$`}`, so that time advances uniformly in all parts of the state.

### 2.2 Nondeterministic Rules and Probabilistic Rewrite Theories

The variables in $v$ and *cond* in a rewrite rule $[l] : u \rightarrow v$ *if cond* are usually a *subset* of the variables $\boldsymbol{x}$ in $u$. However, it is possible to model transitions that exhibit *nondeterministic choice* using rewrite rules of the form $[l] : u(\boldsymbol{x}) \rightarrow v(\boldsymbol{x}, \boldsymbol{y})$ *if* $cond(\boldsymbol{x})$. For each matching substitution $\rho$ of the variables $\boldsymbol{x}$ in $u(\boldsymbol{x})$ that satisfies the condition $cond(\boldsymbol{x})\rho$, the resulting state is not a single state $v(\boldsymbol{x})\rho$, but a possibly infinite family of states $\{v(\boldsymbol{x}, \boldsymbol{y})\rho \uplus \tau\}_\tau$, indexed by the choice of substitution $\tau$ instantiating the extra variables $\boldsymbol{y}$. For example, the rewrite rule $[choose2] : \langle n, m\rangle \rightarrow \langle n, k\rangle$ transforms a pair $\langle n, m\rangle$ of natural numbers into a pair $\langle n, k\rangle$ where $k$ can be *any* natural number. We might choose $k$ according to a *probability distribution* on the natural numbers. This leads to the notion of a *probabilistic rewrite rule*, which is a rewrite rule of the form

$$[l] : u(\boldsymbol{x}) \rightarrow v(\boldsymbol{x}, \boldsymbol{y}) \textit{ if } cond(\boldsymbol{x}) \textit{ with probability } \boldsymbol{y} := \pi(\boldsymbol{x}).$$

For each matching substitution $\rho$ of the variables $\boldsymbol{x}$ in $u(\boldsymbol{x})$ that satisfies the condition $cond(\boldsymbol{x})\rho$, the substitution $\tau$ instantiating the extra variables $\boldsymbol{y}$ is chosen according to a probability distribution $\pi(\boldsymbol{x})\rho$ on the product of data types of the variables $\boldsymbol{y}$, where the probability distribution $\pi(\boldsymbol{x})\rho$ may be parametric on the choice of values $\rho$ instantiating the variables $\boldsymbol{x}$. A *probabilistic rewrite theory* is a rewrite theory where *some* of its rewrite rules are probabilistic.

A nondeterministic rewrite rule $[l] : u(\boldsymbol{x}) \rightarrow v(\boldsymbol{x}, \boldsymbol{y})$ *if* $cond(\boldsymbol{x})$ is *non-executable* (`[nonexec]` in Maude) because a matching substitution $\rho$ of the variables $\boldsymbol{x}$ in $u$ does *not* determine the choice of an extra substitution $\tau$ for the extra

variables $\boldsymbol{y}$ in $v$. By transforming such a nondeterministic rule into a probabilistic rule $u(\boldsymbol{x}) \to v(\boldsymbol{x}, \boldsymbol{y})$ *if* $cond(\boldsymbol{x})$ *with probability* $\boldsymbol{y} := \pi(\boldsymbol{x})$ the rule becomes executable by *simulation* based on *sampling* the probability distribution $\pi(\boldsymbol{x})\rho$ for each matching substitution $\rho$ of the rule's left-hand side.

Even without a probability distribution $\pi(\boldsymbol{x})$, it is still possible to execute nondeterministic rules, which can be useful for *debugging* purposes. Let the nondeterministic variables $\boldsymbol{y} = y_1, \ldots, y_k$ of rule $l$ range over respective sorts $s_1, \ldots, s_k$. Assume that we have added to our theory corresponding sorts $Set[s_1], \ldots, Set[s_k]$ of finite sets of elements in $s_1, \ldots, s_k$, where set union is an associative-commutative binary operation `_,_` with `empty` as identity element. Assume also that we have specified *finite* sets $A_1, \ldots, A_k$ in $Set[s_1], \ldots, Set[s_k]$. Then, our nondeterministic rule $l$ becomes executable in Maude as the rule

$$[l] : u(\boldsymbol{x}) \to v(\boldsymbol{x}, \boldsymbol{y}) \textit{ if } cond(\boldsymbol{x}) \wedge y_1 \textbf{ , } X_1 := A_1 \wedge \ldots \wedge y_k \textbf{ , } X_k := A_k$$

by adding the *matching conditions* [10] $y_i$ **,** $X_i := A_i$, $1 \leq i \leq k$, where the variables $X_1, \ldots, X_k$ range over respective sorts $Set[s_1], \ldots, Set[s_k]$. Only *some* behaviors of the nondeterministic rule can be tested this way, although if $s_1, \ldots, s_k$ are all *finite* data types, then *all* behaviors can be tested in this way.

## 3 The Input Real-Time Rewrite Theories

To support a wide range of applications, we specify a quite general class of real-time rewrite theories—extending timed actor systems—as inputs to the timed $P$ transformation. We describe their basic features in §3.1 and explain additional features in §3.2. Further semantic requirements are discussed in §3.3.

### 3.1 Object-Oriented Real-Time Rewrite Theories

We first explain the basic features of the real-time rewrite theories that are inputs to the timed $P$ transformation. We use *object-oriented* real-time rewrite theories, with local clocks and timers, and whose states include the current configuration (a multiset) of objects, messages, and delayed messages *en route*.

The time domain `Time` denotes the nonnegative real numbers $\mathbb{R}_{\geq 0}$, and its supersort `TimeInf` adds an infinity time value $\infty$ (written `inf` in Maude).

An object may have *multiple timers*. Timers are used to trigger actions: some rule must be applied when a timer expires, i.e., it becomes zero. That rule should either reset the timer to a non-zero natural number, or turn the timer off (i.e., setting it to the infinity value `inf`) while possibly resetting another timer.

The set of timers of an object could change dynamically. For example, in a formalization of the domain name system (DNS), the resolver may have a timer for each user query, e.g., to periodically resubmit an unanswered query to a name server. We define the following datatype `Timers` for sets of timers:

```
sort Timer Timers TimerData .   subsort Timer < Timers .
op [_;_;_] :  TimeInf NzNat TimerData -> Timer [ctor] .
op noTimer : -> Timers [ctor] .
op __ : Timers Timers -> Timers [ctor assoc comm id: noTimer] .
```

The state of a timer is modeled as a term [*timeToExpiration* ; *period* ; *data*], where *timeToExpiration* denotes the expiration time of the timer, *period* denotes the period of the timer, and *data* is the data associated to the timer.

We assume that all the timers of an object are contained in an attribute `timers` of sort `Timers`. An object need not have such an attribute `timers`. An object *may* also have an attribute `clock` of sort `Time`, which models its local clock that is assumed to be perfect. We assume that no object has other time-dependent attributes besides `clock` and/or `timers`.

The timed behavior of the system is given by the following tick rewrite rule:[3]

```
sort DlyMsg .   subsorts Msg < DlyMsg < Configuration .
op dly : Msg Time -> DlyMsg [ctor] .

[tick] : {STATE} => {timeEffect(STATE, T)} in time T
                    if T := mte(STATE) /\ T > 0 [/\ cond] .
```

The function `mte` defines how much time may elapse in a state before a timer expires or a message arrives. Time cannot advance in a state with sending tasks (see below) present, which means that the rule `sendDelayed` in §3.2 must be applied before time advances. The function `timeEffect` decreases the values of all active timers and all message delays, and increases the local clocks, according to the elapsed time. See [21] for the definition of these functions.

### 3.2 Admissible Rewrite Rules in the Input Rewrite Theories

The admissible instantaneous rewrite rules of an input real-time rewrite theory $\mathcal{R}$ to the timed $P$ transformation are described in this section, and satisfy additional requirements described in §3.3. These requirements on $\mathcal{R}$ should ensure that the transformed probabilistic real-time rewrite theory $P(\mathcal{R}, \Pi, init)$ satisfies the *absence of nondeterminism* (AND) property for all states reachable from *init*.

An important novelty, not present in the original $P$ transformation in [17], is that the input theory $\mathcal{R}$ may include several kinds of *nondeterministic rewrite rules* detailed in §3.2.1–3.2.3. This makes the input theories quite general. The reason for the generality afforded by nondeterministic rules is that, as suggested in §2.2, by adding a probability distribution, any nondeterministic rewrite rule becomes a probabilistic one, modeling some random phenomenon.

A main requirement of an input rewrite theory is that only the expiration of a timer or the arrival of a message can trigger an instantaneous rewrite rule.

A key invariant we want to maintain is that, after applying the timed $P$ transformation to an input real-time rewrite theory, in any reachable state of the resulting probabilistic rewrite theory (described in §4), the expiration times of all timers and the arrival times of all messages are all different. To achieve this, an application of a rewrite rule can (re)set at most one timer, to a natural number value, and may turn off other timers. Allowing timers to be (re)set to real numbers could break this invariant, as the following example shows:

[3] We do not show variable declarations, but follow the convention that variables are written in all capital letters.

*Example 1.* When object $A$ receives a *start* message (and it has a local clock), it records the time when the *start* message arrives and starts a timer. $A$ can later send this value as a message to object $B$, which stores this value. When it is time to reset $B$'s timer, it can use this stored "offset of $A$" and set its timer to this offset plus some natural number. Now, the timers of $A$ and $B$ have the same offset, which could lead to them expiring at the same time.

For example, if $A$ receives a message at time 1.23456, it could then set its timer to 100 and send a message to $B$ saying that its timer will expire at time 101.23456. When $B$ receives this message, it can set its own timer to also expire at time 101.23456, which leads to undesirable nondeterminism. □

For the sake of a simpler exposition we exclude rules that create new objects dynamically. However, there should be no problems with generating new objects (with or without timers) as long as at most one timer is (re)set in any rule.

We now describe the admissible rewrite rules in real-time rewrite theories that are inputs to the timed $P$ transformation. As in [17], we assume non-nested objects, i.e., objects that do not contain other objects in some of their attributes.

Besides the *tick* rule in §3.1 we have four types of instantaneous rewrite rules:

1. *Deterministic message-triggered rules* of the form:

   ```
   [l] : (to o [from o'] : mp) < o : Cl | atts >
     =>                        < o : Cl | atts' >  [msgs]  [if cond].
   ```

   or of the form:

   ```
   [l] : (to o [from o'] : mp) < o : Cl | atts >
     =>                        < o : Cl | atts' >    [if cond].
   ```

   if no messages are sent. *If* the object $o$ has a `timers` attribute, the rule can (re)set at most one of its timers, to a natural number if the rule does not generate messages and to a non-zero natural number otherwise, and some timers may by turned off (set to `inf`) or removed. The rule may also add new timers, as long as at most one timer (existing or new) is (re)set.
   In the term `[msgs]` of sort `STask` (for *sending task*) appearing in some of the rules, *msgs* is a term of sort `MsgList` of lists of messages:

   ```
   sorts MsgList STask .  subsort Msg < MsgList .
   subsort STask < Configuration .
   op nil : -> MsgList [ctor] .
   op _;_ : MsgList MsgList -> MsgList [ctor assoc id: nil] .
   op [_] : MsgList -> STask [ctor] .
   ```

2. *Deterministic timer-triggered* rules of the form:

   ```
   [l] : < o : Cl | timers : [0 ; p ; td] otherTimers , atts >
     => < o : Cl | timers : [v ; p' ; td] otherTimers' , atts' > [msgs] [if cond].
   ```

   or of the same form without '`[msgs]`' if no messages are sent. The new (relative) expiration time $v$ of timer *td* is either a non-zero natural number (typically $p$) or `inf`. In the latter case, one other timer in *otherTimers may* be reset to a non-zero natural number.

3. *Nondeterministic network message transmission* rules model the message-sending behavior of a network using the following instantaneous rules:

```
[sendDelayed] : [M ; ML] => dly(M, T)  [ML]  [nonexec] .
[sendNil] : [nil] => none .
```

   These rules evaluate a sending task to a corresponding multiset of $n \geq 0$ of *delayed* messages, where the delay of each message is nondeterministically chosen in the nondeterministic rule `sendDelayed`.
4. *Other nondeterministic* rewrite rules discussed in §3.2.1–3.2.3.

**3.2.1 Modeling Message Arrival and Message Loss.** If the underlying network guarantees that no messages are lost, message arrival is modeled by the rule `[msgArrival] : dly(M,0) => M .` If the network is lossy, message arrival can be modeled by the following *nondeterministic* rewrite rule:

```
[msgArrival] : dly(M,0)  => if B then M else none fi [nonexec] .
```

where `B` is a variable of sort `Bool`, and `none` denotes the empty configuration.

The above rule is *context-free*, which makes it hard to model that the *state* of the network—for example, its congestion—may affect message loss. We can make the `msgArrival` rule *context-sensitive*, to later be able to quantify message loss depending on context, by involving the entire state of objects and messages:

```
[msgArrival] : {dly(M,0) C} => if B then {M C} else {C} fi [nonexec] .
```

where `C` is a variable of sort `Configuration` that matches *the rest of the configuration*, i.e., the *context* in which message `M` is ready to arrive. As explained in §2.2 and §4, the probability distribution associated to a rule in a probabilistic rewrite theory may be *parametric* on the left-hand side's matching substitution. Therefore, having the entire configuration available in `msgArrival` makes it possible for the probability of message loss to vary depending on the context.

In cases where the possibility of messages being lost varies depending on the kinds of messages being sent, instead of a single `msgArrival` rule we may have one such rule for each kind of message by replacing the variable `M` by message expression patterns characterizing each such class.

**3.2.2 Modeling Sensors in Cyber-Physical Systems.** The original $P$ transformation [17] has as input untimed non-probabilistic systems with no nondeterministic rewrite rules. Carolyn Talcott has suggested investigating how to allow input real-time rewrite theories that specify distributed cyber-physical systems with objects that periodically read some sensors in their environment.

This is a pertinent question, since the values read by sensors from their environment are unpredictable, i.e., they correspond to nondeterministic inputs to the system amenable to subsequent quantification by probabilities. That is, the input real-time rewrite theory could model the nondeterminism of sensor inputs, and the timed $P$ transformation could then quantify them probabilistically.

The values read from the environment can be modeled by new, nondeterministic variables in a rule's right-hand side. For example, in a plant the periodic reading of values from its uncertain environment and the corresponding actions can be modeled by an instantaneous nondeterministic rewrite rule of the form:

```
[read-sensor-l] :
   < O : Plant | timers : [0 ; p ; td] TIMERS, values : V, atts >
=> < O : Plant | timers : [p ; p ; td] TIMERS, values : NEW-V, atts' > [msgs]
       [nonexec] .
```

or of the same form without '[*msgs*]', `V` denotes the previous value(s), the nondeterministic variable `NEW-V` describes the "new" value(s) read from the environment at the expiration of the current period, *atts* are the remaining attributes of the *Plant* object, which may change to *atts′* after the transition, and [*msgs*] is the messages to be sent, whose payloads may depend on *atts*, `V`, and `NEW-V`.

**3.2.3 Rules with Finitary Nondeterminism.** Our last type of nondeterministic rules are message-triggered and timer-triggered rewrite rules where the extra variables in their right-hand sides range over some *finite* sorts. We can therefore say that these nondeterministic rules exhibit *finitary nondeterminism*.

Such finitary nondeterminism arises naturally in systems involving objects that, after receiving a message or upon the expiration of a timer, can *choose* from a finite set of *actions*. For example, some objects in an e-commerce system may model *users* who, at various points in the shopping process, may choose among several possible interactions with the system. We consider nondeterministic rules with finitary nondeterminism that the timed $P$ transformation will refine into useful probabilistic rules for many applications.

For the sake of concreteness let us consider a specific class of rules of this kind in systems where one of its finite sorts, say `FinState`, denotes a finite set $Q$ of states of a transition system defined by a function $next : Q \to \mathcal{P}(Q)$ where $q' \in next(q)$ iff $q$ can make a one-step transition to $q'$. Assume that both *next* and the membership predicate $\in$ have been equationally defined. Now consider a class `Cl` with an attribute `finState` with values in `FinState`. Some message-triggered and timer-triggered rules for objects of class `Cl` may exhibit finitary nondeterminism, and may have one of the following respective forms:

```
vars S S'  :  FinState .
[l] : (to o [from o'] : mp) < o : Cl | finState : S, atts >
  =>  l-act[S,S',mp,atts] if S' ∈ next(S) = true [and cond] [nonexec] .
```

and

```
[l] : < o : Cl | finState : S,  timers : [0 ; p ; td] timers,  atts >
  =>  l-act'[S,S',td,timers,atts] if S' ∈ next(S) = true [and cond] [nonexec] .
```

where *l*-`act` is an equationally-defined function such that for each $q, q' \in Q$,

– if $q' \in next(q) = false$, then, *l*-`act`$[q, q', mp, atts]$ = `none`

– if $q' \in next(q) = true$, then,

  $l$-`act[`$q$`,`$q'$`,`$mp$`,`$atts$`] = < `$o$` : Cl | finState : `$q'$`, `$atts\text{-}q.q'$` > [`$msgs\text{-}q.q'$`]`

  or has the same form without '`[`$msgs$`]`' if no messages are sent for the case $q.q'$. All variables appearing in the attributes $atts\text{-}q.q'$ and in $msgs\text{-}q.q'$ must occur in either $mp$ or $atts$.

and where `l-act'` is an equationally-defined function such that for each $q, q' \in Q$,

– if $q' \in next(q) = false$, then, $l$-`act'[`$q$`,`$q'$`,`$td$ $timers$`,`$atts$`] = none`
– if $q' \in next(q) = true$, then,

  $l$-`act'[`$q$`,`$q'$`,`$td$`, `$timers$`,`$atts$`] =`
  `< `$o$` : Cl | finState : `$q'$`, timers : [`$period\text{-}q.q'$` ; `$p$` ; `$td$`] `$timers\text{-}q.q'$`,`
  $atts\text{-}q.q'$ `>  [`$msgs\text{-}q.q'$`]`

  or of the same form without '`[`$msgs$`]`' if no messages are sent for the case $q.q'$. $period\text{-}q.q'$ is either the non-zero natural number $p$ or `inf`. In the latter case, at most one other timer in $timers$ *may* be reset to its non-zero natural number period. All variables appearing in $timers\text{-}q.q'$, $atts\text{-}q'$, and $msgs\text{-}q.q'$ must appear in $timers$ or $atts$.

### 3.3 Requirements on Input Theories and Initial States

An initial state `{`$initconf$`}`, where $initconf$ is a ground term of sort `Configuration`, of a real-time rewrite theory in our input class consists of a set of objects with distinct names and an optional sending task, with a list of messages to be sent with undetermined delays, of the form

`< `$o_1$` : `$C_1$` | `$atts_1$` >  ...  < `$o_n$` : `$C_n$` | `$atts_n$` >`
`[(to `$o_{i_1}$` [from `$o_{l_1}$`] : `$mp_{i_1}$`) ; ... ; (to `$o_{i_k}$` [from `$o_{l_k}$`] : `$mp_{l_k}$`)]`

such that the value of each local `clock` attribute is `0`. The rewrite theory and the initial configuration $initconf$ together satisfy the following requirements:

1. At most one timer in $initconf$ in a single object $o_j$ is turned on with a timer value a natural number, which cannot be 0 unless the initial state does not have a sending task. Indeed, if there is no sending task a single object $o_j$ must be turned on, since otherwise $initconf$ would be a deadlock state.
2. Any object *enabled* by a message-triggered rule to receive a given message addressed to it, is so enabled by the application of a *unique* message-triggered rule with a *unique* substitution of its left-hand side variables.
3. Any object with a single expired timer and enabled to perform a transition by a timer-triggered rule (including such rules with finitary nondeterminism and sensor-reading rules), is so enabled by a *unique* rule among those for that timer, with a *unique* substitution of its left-hand side variables.

4. Any (message-triggered or timer-triggered) transition may set the value of at most *one* existing or new (added by the rule) timer to a natural number, and may turn off some other timers. If the transition was triggered by a timer or the transition generates a sending task, then the reset timer must be reset to a non-zero natural number; furthermore, if the set timer is not the one that expired, then the expired timer must be turned off.
5. All addresses of all messages in the initial states or generated by the timer- and message-triggered rules are names of objects in the initial configuration.

## 4 The Timed $P$ Transformation

The *timed $P$ transformation* transforms an input real-time rewrite theory and an initial state satisfying the requirements in §3 by using a family $\Pi$ of parametric probability distributions for:

- delays of messages,
- probability of message losses (if the network is lossy),
- the values read by sensors in "sensor rules" for cyber-physical systems, and
- choosing the next "state" $q'$ in any rule with finitary nondeterminism

into a probabilistic rewrite theory. Formally, the timed $P$ transformation is a mapping $P : (\mathcal{R}, init, \Pi) \mapsto (\mathcal{R}_\Pi, init_\Pi)$ where:

- $\mathcal{R} = (\Sigma, E, L, R)$ is a real-time rewrite theory meeting the input requirements in §3, where all the rules have distinct labels, and where $\texttt{init} \notin L$.
- *init* is an initial system state satisfying the requirements in §3.3.
- $\Pi$ is a partial function from the set $L \cup \{\texttt{init}\}$ to parametric probability distributions or, in some cases, pairs and/or families of such distributions. $\Pi$ is defined as follows:
  1. For a message-triggered or timer-triggered rule $l$ that is not a rule with finitary nondeterminism or a sensor rule, and that has a sending task in its right-hand side, $\Pi$ has a map $l \mapsto \pi_l(\boldsymbol{x})$, where $\boldsymbol{x}$ are the variables in the left-hand side of rule $l$ and $\pi_l(\boldsymbol{x})$ is a parametric *continuous* probability distribution on the non-negative reals $\mathbb{R}_{\geq 0}$, defining the probability distribution over the delays of the messages generated by the rule $l$ as a function of the state of the sending object and (for message-triggered rules) the received message. Further assumptions on the message delay distributions $\pi_l(\boldsymbol{x})$ are detailed in §4.4.
  2. When a lossy network is assumed, the mapping in $\Pi$ for the rule label `msgArrival` is $\texttt{msgArrival} \mapsto \pi_{\texttt{msgArrival}}(\boldsymbol{x})$, where $\boldsymbol{x}$ are the variables in the rule's left-hand side and $\pi_{\texttt{msgArrival}}(\boldsymbol{x})$ is a parametric probability distribution on the Booleans, defining the probability that a message is lost as a function of the message or of the message and the entire state (depending on which version of the rule `msgArrival` is used).
  3. For a sensor-reading rule `read-sensor-`$l$ that generates a message-sending task and whose left-hand side variables are $\boldsymbol{x}$, the mapping in $\Pi$ is

$$\texttt{read-sensor-}l \mapsto (\pi_{\texttt{read-sensor-}l.val}(\boldsymbol{x}), \pi_{\texttt{read-sensor-}l}(\boldsymbol{x}))$$

where $\pi_{\texttt{read-sensor-}l.val}(\boldsymbol{x})$ is a parametric probability distribution on the data type of `Values`, and $\pi_{\texttt{read-sensor-}l}(\boldsymbol{x})$ is a parametric continuous probability distribution on $\mathbb{R}_{\geq 0}$. If the rule does not generate a sending task, then the mapping is just `read-sensor-`$l \mapsto \pi_{\texttt{read-sensor-}l.val}(\boldsymbol{x})$.

4. For a rule $l$ with finitary nondeterminism that uses a finite transition system $\mathcal{Q} = (Q, \rightarrow_{\mathcal{Q}})$, and with left-hand side variable `S` of sort `FinState`, the mapping in $\Pi$ has the form

$$l \mapsto (\pi_{l.\mathcal{Q}}(S), \{\pi_{l_{q.q'}}(\boldsymbol{x} \setminus \{\texttt{S}\})\}_{q \in Q, q' \in next(q) \wedge generatesMsgs(q,q')})$$

where $\boldsymbol{x}$ are the variables in the rule's left-hand side. $\pi_{l.\mathcal{Q}}(S)$ is the parametric probability distribution $\pi_{l.\mathcal{Q}}(S) = \{\pi_{l.\mathcal{Q}}(S)\}_{S \in Enabled(Q)}$, where $Enabled(Q)$ denotes the complement of the set of terminating states in $\mathcal{Q}$ and each $\pi_{l.\mathcal{Q}}(q)$ is a *discrete* probability distribution on $next(q)$ in $\mathcal{Q}$. Therefore, $\pi_{l.\mathcal{Q}}(S)$ defines a *Markov chain* quantifying the nondeterminism of the transition system $\mathcal{Q}$. $\{\pi_{l_{q.q'}}(\boldsymbol{x} \setminus \{\texttt{S}\})\}_{q \in Q, q' \in next(q) \wedge generatesMsgs(q,q')}$ is a family of parametric continuous probability distributions over $\mathbb{R}_{\geq 0}$ providing probability distributions for the delays of the messages generated when the rule $l$ is applied in "state" $q$ and chooses $q'$ as its next state, but only for those $q.q'$-pairs where a sending task is generated.

5. Finally, $\Pi$ contains a map `init` $\mapsto \pi_{\texttt{init}}$, where $\pi_{\texttt{init}}$ is a continuous probability distribution on $\mathbb{R}_{\geq 0}$ defining the probability distribution of the delays of the messages in the initial state.

– $\mathcal{R}_\Pi = (\Sigma_\Pi, E_\Pi, L_\Pi, R_\Pi)$ is the probabilistic rewrite theory defined below.
– $init_\Pi$ is the timed $P$-transformed initial state defined in §4.3.

In what follows, we define in detail the probabilistic theory $\mathcal{R}_\Pi$ and the initial state $init_\Pi$ obtained by applying the timed $P$ transformation to $(\mathcal{R}, init, \Pi)$.

### 4.1 The Equational Theory $(\Sigma_\Pi, E_\Pi)$

The equational theory $(\Sigma_\Pi, E_\Pi)$ extends $(\Sigma, E)$, and also contains the functions required to define the continuous probability distributions in $\Pi$. The timed $P$ transformation removes the subsort declaration `STask < Configuration` and adds a new sort `LSTask` (for "labeled sending task") as a subsort of `Configuration`.

The $P$ transformation also adds, for each rule $l$ that may generate new messages and whose left-hand side variables are $\boldsymbol{x} = x_1 : s_1, \ldots, x_n : s_n$, an operator

```
op msgDly_l : s_1 ... s_n STask -> LSTask [ctor] .
```

However, if the rule $l$ is a rule with finitary nondeterminism whose left-hand side has variables `S` and $\boldsymbol{x} = x_1 : s_1, \ldots, x_n : s_n$, then we instead add an operator

```
op msgDly_{l_{q.q'}} : s_1 ... s_n STask -> LSTask [ctor] .
```

for each transition $q \rightarrow q'$ "in" $l$ that may generate messages. We also declare the function `op msgDly`$_{\texttt{init}}$ `: STask -> LSTask [ctor]` used for probabilistically quantifying the delays in sending tasks for initial states.

The equations for $l$-`act` and $l$-`act'` are also modified, as explained under the treatment of rules with finitary nondeterminism in §4.2.

The equations for `mte` and `timeEffect` are modified by removing the equations for sending tasks, and adding the equations

```
var LSTASK : LSTask .  eq mte(LSTASK) = 0 .  eq timeEffect(LSTASK, T) = LSTASK .
```

### 4.2 The Rewrite Rules $R_\Pi$

This section explains how each kind of the rewrite rules § 3.2 is transformed by the timed $P$ transformation, which leaves the tick rule in §3.1 unchanged.

*Deterministic Message-Triggered Rewrite Rules.* The timed $P$ transformation transforms each deterministic message-triggered rewrite rule in § 3.2 that generates a sending task into the corresponding rewrite rule

```
[l] : (to o [from o'] : mp) < o : Cl | atts >
    =>                      < o : Cl | atts' >  msgDly_l(x, [msgs]) [if cond].
```

where $\boldsymbol{x}$ are the variables in the left-hand side of rule $l$. Deterministic message-triggered rules that do not generate new messages are not modified by $P$.

*Deterministic Timer-Triggered Rules.* Each deterministic timer-triggered Rule $l$ that may generate messages is transformed by $P$ into the timer-triggered rule

```
[l] : < o : C | timers : [0 ; p ; td] otherTimers, atts >
  => < o : C | timers : [v ; p' ; td] otherTimers, atts' > msgDly_l(x, [msgs]) [if cond] .
```

Such rules that do not generate messages are not modified by $P$.

*Network Transmission Rules.* $P$ transforms the rule `sendDelayed` into a *family* of *probabilistic* rewrite rules parameterized by those rule labels $l \in L$ that are in the domain of $\Pi$ and where $l$ generates a message task as follows. If $l$ is not a rule with finitary nondeterminism, then the corresponding rule is:

```
[sendDelayed_l] :
 msgDly_l(x, [M ; ML]) => dly(M, T) msgDly_l(x, [ML]) with probability T := π_l(x) .
```

If $l$ labels a rule with finitary nondeterminism, there are as many probabilistic rules as transitions $q \to q'$ such that when $l$ changes $q$ to $q'$ some messages can be sent. Each such probabilistic rule has the form:

```
[sendDelayed_{l_{q.q'}}] :
msgDly_{l_{q.q'}}(x, [M ; ML]) => dly(M,T) msgDly_{l_{q.q'}}(x,[ML]) with probability T := π_{l_{q.q'}}(x) .
```

The rule `sendNil` is transformed by $P$ into `[sendNil_l] : msgDly_l(x,[nil]) => none` for each $l$ as above, and a similar rule for each "message-generating" $l_{q.q'}$. If the initial state contains a sending task, $R_\Pi$ also contains the rewrite rules

```
[sendDelayed_init] :
 msgDly_init([M ; ML]) => dly(M, T) msgDly_init([ML]) with probability T := π_init .
[sendNil_init] : msgDly_init([nil]) => none .
```

*Message Arrival and Loss.* There are three versions of the rule `msgArrival`. $P$ leaves the first unchanged, and transforms either of the other two into the corresponding *probabilistic* rewrite rule

```
[msgArrival] :
 dly(M,0) => if B then M else none fi with probability B := π_msgArrival(M) .
    or
[msgArrival] :
{dly(M,0) C}  => if B then {M C} else {C} fi
                 with probability B := π_msgArrival(M,C) .
```

*Sensor Rules.* The timed $P$ transformation transforms each *sensor* rule `read-sensors-`$l$ that generates messages into the *probabilistic* rewrite rule

```
[read-sensor-l] :
   < o : Plant | values : V,     timers : [0 ; P ; td] TIMERS, atts >
=> < o : Plant | values : NEW-V, timers : [P ; P ; td] TIMERS, atts' >
   msgDly_{read-sensors-l}(x, [msgs]) [if cond]
       with probability NEW-V := π_{read-sensors-l.val}(x) .
```

where $\boldsymbol{x}$ are the variables in the left-hand side of the rule, which include the variable `V` for the previously read sensor value. A sensor rule that does not generate messages is transformed in a similar way by $P$.

*Rules with Finitary Nondeterminism.* $P$ transforms a message-triggered rule $l$ with finitary nondeterminism in §3.2.3 into the *probabilistic* rewrite rule

```
[l] : (to o from o' : mp)  < o : C | finState: S, atts >
 =>   l-act[S,S',mp,atts]  [if cond]  with probability S' := π_{l.Q}(S) .
```

and transforms the *equations* involving sending tasks into the equations

```
eq l-act[q,q',atts] = < o : C | finState : q', atts-q' > msgDly_{l_{q.q'}}(x,[msgs-q.q'])
```

where $\boldsymbol{x}$ are the variables in the rule $l$ except for `S`, which are present in *atts* in this equation. If the above equation does not have a sending task ([*msgs-q.q'*]) in its right-hand side, then the equation is not modified by $P$. The timed $P$ transformation is similar for timer-triggered rules with finitary nondeterminism.

### 4.3 The Initial State $init_{\Pi}$

The timed $P$ transformation transforms an initial state *init* of the form {*objects* [*messageList*]} into the initial state $init_{\Pi}$, which is the term {*objects* $\texttt{msgDly}_{\texttt{init}}$([*messageList*])}.

### 4.4 Assumptions on the Message Delay Distributions in $\Pi$

The message delay distributions $\pi_l(\boldsymbol{x}_l)$ are continuous distributions on the non-negative reals $\mathbb{R}_{\geq 0}$ parametric on the left-hand side variables $\boldsymbol{x}_l$ of rule $l$. Likewise, the distribution $\pi_{\texttt{init}}$ is a continuous distribution on $\mathbb{R}_{\geq 0}$. These continuous distributions model the network's probabilistic behavior when sending the messages. The following assumptions from [17] will be used:

1. For each message- and timer-triggered rule $l$ that generates messages and each ground substitution $\theta = \{\boldsymbol{x}_l \mapsto \boldsymbol{a}\}$, instantiating the parameters $\boldsymbol{x}_l$ of $l$'s left-hand side (resp. for $l = init$) there is a piecewise continuous *probability density function* $f_l(\boldsymbol{a}) : \mathbb{R} \to [0, +\infty)$ (resp. $f_{init}$) such that—since message delays are always non-negative—for $x < 0$, $f_l(\boldsymbol{a})(x) = 0$ (resp. $f_{init}(x) = 0$).
2. Each such density function defines a *probability distribution function* $\pi_l(\boldsymbol{a}) : \mathbb{R} \to [0, 1]$ (resp. $\pi_{init} : \mathbb{R} \to [0, 1]$) as a continuous and almost everywhere differentiable function of the form $\pi_l(\boldsymbol{a}) = \lambda x \in \mathbb{R}.\ \int_{-\infty}^{x} f_l(\boldsymbol{a})(t)\ dt$ (likewise for $\pi_{init}$). By the assumptions on $f_l(\boldsymbol{a})$ and $f_{init}$, $\pi_l(\boldsymbol{a})(x) = 0$ and $\pi_{init}(x) = 0$, for $x \leq 0$. More generally, yet equivalently ([16], Theorem 1.88), $f_l(\boldsymbol{a})$ defines a *probability measure* $\mu_l(\boldsymbol{a}) : \mathcal{B}(\mathbb{R}) \to [0, 1]$ on the Borel $\sigma$-algebra of $\mathbb{R}$, defined by $\mu_l(\boldsymbol{a}) = \lambda B \in \mathcal{B}(\mathbb{R}).\ \int_{t \in B} f_l(\boldsymbol{a})(t)\ dt$.
3. In what follows, $X_{f_l(\boldsymbol{a})}$, which we call the *support* of $f_l(\boldsymbol{a})$, will denote the set $X_{f_l(\boldsymbol{a})} = \{x \in \mathbb{R} \mid f_l(\boldsymbol{a})(x) > 0\}$, and $\overline{X}_{f_l(\boldsymbol{a})}$ will denote its topological closure, obtained by adding to $X_{f_l(\boldsymbol{a})}$ its limit points. The set $\overline{X}_{f_l(\boldsymbol{a})}$ provides a useful "envelope" for the values obtained by sampling $\pi_l(\boldsymbol{a})$. Indeed, by openness, for any $x \in \mathbb{R} \setminus \overline{X}_{f_l(\boldsymbol{a})}$ there is an open interval $(a, b) \subseteq \mathbb{R} \setminus \overline{X}_{f_l(\boldsymbol{a})}$ with $x \in (a, b)$, so that $\mu_l(\boldsymbol{a})((a, b)) = \pi_l(\boldsymbol{a})(b) - \pi_l(\boldsymbol{a})(a) = 0$. Therefore, any real number $r$ obtained by sampling the distribution $\pi_l(\boldsymbol{a})$ must be inside the envelope $\overline{X}_{f_l(\boldsymbol{a})}$. Notice, furthermore, that the assumption on $f_l(\boldsymbol{a})$ and $f_{init}$ forces the inclusions $\overline{X}_{f_l(\boldsymbol{a})} \subseteq [0, +\infty)$ and $\overline{X}_{f_{init}} \subseteq [0, +\infty)$.

## 5 Correctness of the Timed $P$ Transformation

The timed $P$ transformation maps a real-time rewrite theory $\mathcal{R}$ meeting $P$'s input requirements into the probabilistic rewrite theory $\mathcal{R}_\Pi$. The purpose of $\mathcal{R}_\Pi$ is to both *quantify* and perhaps *restrict* the nondeterministic behaviors of $\mathcal{R}$. But is $P$ correct? Two key correctness requirements are that: (1) the *behaviors* of $\mathcal{R}_\Pi$ are always semantically related to those of $\mathcal{R}$; this is proved in §5.1; and (2) $\mathcal{R}_\Pi$ is a *purely probabilistic system* and can therefore be formally analyzed by statistical model checking; this is proved in §5.2.

### 5.1 Faithful Behavioral Correspondence between $\mathcal{R}_\Pi$ and $\mathcal{R}$

$\mathcal{R}_\Pi$ is a non-executable mathematical model. To talk about its "behaviors" we need to *simulate* $\mathcal{R}_\Pi$ by *sampling* the probability distributions in its probabilistic rewrite rules. But there is a simpler way of comparing $\mathcal{R}$ and $\mathcal{R}_\Pi$ than via

sampling: we can use the so-called *nondeterministic envelope* $NdEnv(\mathcal{R}_\Pi)$ of $\mathcal{R}_\Pi$, a nondeterministic real-time rewrite theory that contains all the behaviors that could be obtained by sampling $\mathcal{R}_\Pi$ without any need to bring sampling techniques into the picture. Due to space limitations we sketch $NdEnv(\mathcal{R}_\Pi)$'s construction and refer to [21] for full details. The key idea is to replace each probabilistic rule in $\mathcal{R}_\Pi$ by a nondeterministic rule such that its choices of values are those obtainable by sampling the probabilistic rule's distribution. For example, each rule of the form `sendDelayed`$_l$ is replaced by the nondeterministic rule:

```
[sendDelayed_l] :
msgDly_l(x, [M ; ML]) => dly(M,T) msgDly_l(x, [ML]) if T ∈ X̄_{f_l}(x) [nonexec] .
```

where, as explained in §4.4, $\overline{X}_{f_l}(\boldsymbol{a})$ is the *envelope* for the values obtained by sampling the distribution $\pi_l(\boldsymbol{a})$ for each instantiation of $\boldsymbol{x}$ to ground values $\boldsymbol{a}$.

We can relate the behaviors of $NdEnv(\mathcal{R}_\Pi)$ and those of $\mathcal{R}$ by means of a *simulation map* $h : NdEnv(\mathcal{R}_\Pi) \to \mathcal{R}$, which, by definition, is a function $h$ mapping states of $NdEnv(\mathcal{R}_\Pi)$ to states of $\mathcal{R}$ in such a way that, for any state $u$ of $NdEnv(\mathcal{R}_\Pi)$ that can perform a rewrite transition $u \to u'$ in $NdEnv(\mathcal{R}_\Pi)$, the state $h(u)$ can perform a rewrite transition $h(u) \to h(u')$ in $\mathcal{R}$. The fact that qualitative properties of $\mathcal{R}$ and the quantitative properties of $\mathcal{R}_\Pi$ are properties of the *same* system is ensured by the function $h$ defined in [21] that, by the theorem below, is a simulation map as proved in [21].

**Theorem 1.** *The function $h$ defines a simulation map of rewrite theories $h : NdEnv(\mathcal{R}_\Pi) \to \mathcal{R}$.*

### 5.2 Absence of Nondeterminism (AND)

To analyze the quantitative properties of $\mathcal{R}_\Pi$ by statistical model checking it is highly desirable that the sampling-based simulation of $\mathcal{R}_\Pi$ from an initial state $init_\Pi$ is a *purely probabilistic system*. This just means that for any state reachable from $init_\Pi$ at most one rewrite transition (probabilistic or not) is possible, since otherwise the system would exhibit a form of nondeterminism. In more detail, this means that the following *absence of nondeterminism* (AND) property holds:

**Definition 1.** *(AND) [17]. With probability 1, for any state reachable from $init_\Pi$ by a sequence of sampling-based rewrite steps (i.e., by a Monte Carlo simulation of $\mathcal{R}_\Pi$), there is at most one rewrite rule applicable, at a unique position and with a unique matching substitution $\theta$ for its left-hand side; i.e., two different rules, or the same rule, can never be applied to a reachable state either at different positions or with different matching substitutions.*

The proof of the following theorem can be found in [21].

**Theorem 2.** *If the real-time rewrite theory $\mathcal{R}$ and the initial state init satisfy the input requirements of the timed $P$ transformation, then $\mathcal{R}_\Pi$ from initial state $init_\Pi$ satisfies the AND property.*

## 6 A Design and Analysis Methodology for Distributed Real-Time Systems

As this work has shown, a distributed real-time system design expressed as a real-time rewrite theory often includes nondeterministic rules that are non-executable. This poses challenges for debugging such designs *prior* to analyzing their quantitative properties, since the design may contain bugs and design flaws that are hard to uncover without testing the system. Furthermore, verification of qualitative properties, such as safety properties, may also be important. This section outlines a design and analysis methodology addressing these challenges:

1. **Design Capture** of the intended system as a real-time rewrite theory $\mathcal{R}$.
2. **Execution and Testing** of $\mathcal{R}$ following the methodology outlined in §2.2: By choosing finite subsets of values for nondeterministic variables, all rules can become executable by adding matching conditions to the rules to select specific values in the domain. After testing and debugging $\mathcal{R}$ through execution, Maude's `search` command can exhaustively explore all behaviors from an initial state that satisfy the restrictions imposed by the choices of values.
3. **Symbolic Execution and Reachability Analysis**. The original specification $\mathcal{R}$, while non-executable, is *symbolically* executable by *rewriting modulo SMT* [25]. The idea is that we can symbolically execute, not concrete states, but symbolic ones of the form $u(\boldsymbol{x}) \mid \varphi(\boldsymbol{x})$ where $\varphi(\boldsymbol{x})$ is a formula in a decidable theory constraining the values of the variables $\boldsymbol{x}$. For example, some of those variables may range over the real numbers, and $\varphi(\boldsymbol{x})$ may contain constraints over those variables in an SMT-decidable theory such as linear arithmetic. Rewriting modulo SMT makes it possible to verify *safety properties* by reachability analysis. For formal analyses of real-time systems using rewriting modulo SMT, see [6,5,7].
4. **Simulation of** $Sim(\mathcal{R}_{\Pi})$. The *Sim* transformation [17] makes the probabilistic rewrite theory $\mathcal{R}_{\Pi}$ executable by *sampling* the probability distribution of each rule. This allows users to *simulate* the probabilistic behavior of $\mathcal{R}_{\Pi}$.
5. **Statistical Model Checking of** $Sim(\mathcal{R}_{\Pi})$. Thanks to Theorem 2, it is possible to analyze the behaviors of $\mathcal{R}_{\Pi}$ by *statistical model checking*. This can be done using, e.g., the QMaude tool [26] for properties expressed in the QuaTEx probabilistic temporal logic.

## 7 Related Work

The timed $P$ transformation extends the $P$ transformation in [17], whose inputs are *untimed* generalized actor rewrite theories. The (timed) input theories in this paper support timers and message delays, which make no sense for untimed models. The input theories in [17] do not support nondeterminism in the rules themselves, whereas we support nondeterministic sensor rules, rules with finitary nondeterminism, and nondeterministic rule(s) for message loss/delivery. On the other hand, the input theories in [17] support support both message-triggered

rules (which we also do) and more general "object-triggered rules," whereas in this work we focus on *timer-triggered* rules, which are the kind of object-triggered rules most typical in real-time systems.

Lingua Franca [18] and Timed Rebeca [1] are two well-known timed actor languages. Lingua Franca also supports both message-triggered and timer-triggered (re)actions. Lingua Franca provides "deterministic concurrency:" the only source of nondeterminism are external "physical actions," which can take place at "any" time and have any value. In contrast to our input theories, the delay of all messages between the same pair of ports is the same, timer periods do not change, and there are no message losses. On the other hand, the time- and value-nondeterministic physical actions in Lingua Franca are only indirectly supported by our theories, in which such an action can be simulated by sending a message with nondeterministic delay, setting a sensor timer to 0 when it arrives, which triggers a `read-sensor` rule to obtain a value nondeterministically.

Timed Rebeca [1] extends the actor language Rebeca [28] to discrete real-time systems. Timed Rebeca supports delay statements, messages with delay intervals, and nondeterministic assignment of state variables to any value in a finite set. Unlike in our input theories, the following are not supported: timers (although periodic behavior can be simulated by an actor sending a message to itself); "sensor rules" sampling infinite domains; message loss; and dense time.

The main contribution of our paper is to define a transformation that maps a nondeterministic timed actor system into a purely probabilistic timed one, and proving a semantic relationship between the two models.

Although probabilistic timed versions of Rebeca exist [15,14], statistical model checking is only supported to choosing the next state and assign variable values from discrete distributions. Continuous distributions, important for performance analysis, are not supported. The main contrast with our paper is that there is no work mapping a Timed Rebeca model to a probabilistic timed model.

We are not aware of any probabilistic version of Lingua Franca, or of any mapping from Lingua Franca programs to probabilistic ones.

Other pairs of related nondeterministic and probabilistic formalisms are discussed in [17]; examples include timed [4] and probabilistic and stochastic timed automata [12], and BIP [8] and $\mathcal{S}$BIP [22]. However, we are unaware of any work mapping qualitative timed distributed systems to quantitative ones.

Recent work [23] by Olarte et al. provides a "probabilistic" Maude semantics to probabilistic Event-B to support statistical model checking of such Event-B models using `umaudemc`. Like our finitary nondeterminism and sensor rules, their model supports "external inputs [...] uniformly chosen from finite sets" and "probabilistic assignments to variables [which] model uncertainty in 'external' data." The differences between our finitary nondetermism and sensor rules are that [23] only supports non-parametric *uniform* discrete distributions, and does not support user-defined (discrete or continuous) distributions which, furthermore, in our case could be parametric on the current state. The main difference between our work and the work in [23] is that the timed $P$ transformation takes as inputs and yields as outputs *timed* systems, whereas all models in [23] are un-

timed (and therefore do not involve timers and message delays). We prove that our resulting models are fully probabilistic "by construction," whereas Olarte et al. obtain fully probabilistic models by adding (discrete) weights to the different events, akin to how weights are added to rewrite rules (and substitutions and contexts) in QMaude [26] and PSMaude [9] to obtain fully probabilistic models.

## 8 Concluding Remarks

This work is a further step in the effort initiated in [17] to bridge the semantic gap between the different models used to verify qualitative and quantitative properties of distributed systems. In [17] this was done for *untimed* systems. The main contribution of this paper has been to extend the $P$ transformation to a general class of distributed *real-time* systems that extends timed actor systems. The *soundness* of the quantitative analysis of distributed real-time systems supported by our *timed* $P$ transformation crucially depends on its correctness. We have proved two key correctness results. The first proves the semantic consistency between the input real-time rewrite theory $\mathcal{R}$ and its $P$-transformed theory $\mathcal{R}_{\Pi}$, thus ensuring that $\mathcal{R}$ and $\mathcal{R}_{\Pi}$ are models of the *same* system at different levels of abstraction. The second shows that $\mathcal{R}_{\Pi}$ is a purely probabilistic model and makes possible the automated quantitative analysis of $\mathcal{R}_{\Pi}$ by statistical model checking. We have also discussed in §6 how our timed $P$ transformation supports a *formal design and verification methodology* for distributed real-time systems.

Much work remains ahead to bring such a design and verification methodology to fruition. First of all, the *automation* of the timed $P$ transformation needs to be implemented. Our goal is to make this transformation available for users of the QMaude tool [26]. This will greatly facilitate the access to QMaude by real-time distributed system designers. Second, this should be done in conjunction with substantial experimentation, resulting in a suite of case studies demonstrating the usefulness of the timed $P$ transformation and its supporting tools in the design and both qualitative and quantitative verification of distributed systems.

*Acknowledgments.* We are grateful to Joud Khoury, Minyoung Kim, Narciso Martí-Oliet, Christophe Merlin, Carolyn Talcott, and the anonymous reviewers for helpful comments on an earlier version of this paper, and to Rubén Rubio for his help with the QMaude tool. This material is based upon work supported by the Defense Advanced Research Projects Agency (DARPA) under Contract No HR001125CE019.

## References

1. Aceto, L., Cimini, M., Ingólfsdóttir, A., Reynisson, A.H., Sigurdarson, S.H., Sirjani, M.: Modelling and simulation of asynchronous real-time systems using Timed Rebeca. In: FOCLASA. EPTCS, vol. 58, pp. 1–19 (2011)
2. Agha, G.: Actors. MIT Press (1986)

3. Agha, G., Palmskog, K.: A survey of statistical model checking. ACM Trans. Model. Comput. Simul. **28**(1), 6:1–6:39 (2018)
4. Alur, R., Dill, D.L.: A theory of timed automata. Theoretical Computer Science **126**(2), 183–235 (1994)
5. Arias, J., Bae, K., Olarte, C., Ölveczky, P.C., Petrucci, L.: A rewriting-logic-with-SMT-based formal analysis and parameter synthesis framework for parametric time Petri nets. Fundamenta Informaticae **192**(3-4), 261–312 (2024)
6. Arias, J., Bae, K., Olarte, C., Ölveczky, P.C., Petrucci, L., Rømming, F.: Symbolic analysis and parameter synthesis for networks of parametric timed automata with global variables using Maude and SMT solving. Science of Computer Programming **233**, 103074 (2024)
7. Bae, K., Olarte, C., Ölveczky, P.C.: Modeling and analyzing real-time systems in rewriting logic. In: Concurrent Programming, Open Systems and Formal Methods: Essays Dedicated to Gul Agha to Celebrate His Scientific Career. pp. 494–535. Springer Nature Switzerland, Cham (2026)
8. Basu, A., Bensalem, S., Bozga, M., Combaz, J., Jaber, M., Nguyen, T., Sifakis, J.: Rigorous component-based system design using the BIP framework. IEEE Softw. **28**(3), 41–48 (2011)
9. Bentea, L., Ölveczky, P.C.: A probabilistic strategy language for probabilistic rewrite theories and its application to cloud computing. In: Proc. WADT 2012. pp. 77–94. Lecture Notes in Computer Science, Springer (2012)
10. Clavel, M., Durán, F., Eker, S., Meseguer, J., Lincoln, P., Martí-Oliet, N., Talcott, C.: All About Maude – A High-Performance Logical Framework. Springer LNCS Vol. 4350 (2007)
11. David, A., Larsen, K.G., Legay, A., Mikucionis, M., Poulsen, D.B.: Uppaal SMC tutorial. Int. J. Softw. Tools Technol. Transf. **17**(4), 397–415 (2015)
12. David, A., Larsen, K.G., Legay, A., Mikucionis, M., Poulsen, D.B.: Uppaal SMC tutorial. Int. J. Softw. Tools Technol. Transf. **17**(4), 397–415 (2015)
13. Goguen, J., Meseguer, J.: Order-sorted algebra I: Equational deduction for multiple inheritance, overloading, exceptions and partial operations. Theoretical Computer Science **105**, 217–273 (1992)
14. Jafari, A., Khamespanah, E., Kristinsson, H., Sirjani, M., Magnusson, B.: Statistical model checking of Timed Rebeca models. Comput. Lang. Syst. Struct. **45**, 53–79 (2016)
15. Jafari, A., Khamespanah, E., Sirjani, M., Hermanns, H., Cimini, M.: PTRebeca: Modeling and analysis of distributed and asynchronous systems. Science of Computer Programming **128**, 22–50 (2016)
16. Klenke, A.: Probability Theory. Springer (2006)
17. Liu, S., Meseguer, J., Ölveczky, P.C., Zhang, M., Basin, D.A.: Bridging the semantic gap between qualitative and quantitative models of distributed systems. Proc. ACM Program. Lang. **6**(OOPSLA2), 315–344 (2022)
18. Lohstroh, M., Menard, C., Bateni, S., Lee, E.A.: Toward a Lingua Franca for Deterministic Concurrent Systems. ACM Transactions on Embedded Computing Systems (TECS) **20**(4), 1–27 (2021)
19. Meseguer, J.: Conditional rewriting logic as a unified model of concurrency. Theoretical Computer Science **96**(1), 73–155 (1992)
20. Meseguer, J.: Twenty years of rewriting logic. J. Algebraic and Logic Programming **81**, 721–781 (2012)
21. Meseguer, J., Ölveczky, P.C.: The timed $P$ transformation for real-time distributed systems (longer report) (2026), http://olveczky.se/timed-p-techrep.pdf

22. Nouri, A., Mediouni, B.L., Bozga, M., Combaz, J., Bensalem, S., Legay, A.: Performance evaluation of stochastic real-time systems with the SBIP framework. Int. J. Crit. Comput. Based Syst. **8**(3/4), 340–370 (2018)
23. Olarte, C., Osorio, D., Ramírez, C., Rocha, C.: Algorithmic analysis of Event-B in rewriting logic. In: Proc. NASA Formal Methods. pp. 275–293. Springer Nature Switzerland, Cham (2025)
24. Ölveczky, P.C., Meseguer, J.: Specification of real-time and hybrid systems in rewriting logic. Theoretical Computer Science **285**(2), 359–405 (2002)
25. Rocha, C., Meseguer, J., Muñoz, C.A.: Rewriting modulo SMT and open system analysis. Journal of Logic and Algebraic Methods in Programming **86**, 269–297 (2017)
26. Rubio, R., Martí-Oliet, N., Pita, I., Verdejo, A.: QMaude: Quantitative specification and verification in rewriting logic. In: Chechik, M., Katoen, J., Leucker, M. (eds.) Formal Methods - 25th International Symposium, FM 2023, Lübeck, Germany, March 6-10, 2023, Proceedings. Lecture Notes in Computer Science, vol. 14000, pp. 240–259. Springer (2023)
27. Rubio, R., Riesco, A., Martí-Oliet, N.: PMaude revisited through probabilistic strategies. In: Concurrent Programming, Open Systems and Formal Methods: Essays Dedicated to Gul Agha to Celebrate His Scientific Career. pp. 475–493. Lecture Notes in Computer Science, Springer (2026)
28. Sirjani, M., Movaghar, A., Shali, A., de Boer, F.S.: Modeling and verification of reactive systems using Rebeca. Fundamenta Informaticae **63**(4), 385–410 (2004)